

SKILLSHARE HUB

**Submitted in partial fulfilment of the requirements for the award
of the degree of**

**BACHELOR OF SCIENCE
IN
INFORMATION TECHNOLOGY**

Submitted by

Jagan.B

TABLE OF CONTENTS

CHAPTER NO.	CHAPTER TITLE	PAGE NO.
1.	PROBLEM STATEMENT	
2.	SYSTEM STUDY / LITERATURE SURVEY (i) REVIEW ON THE EXISTING WORK (ii) PROPOSED WORK (iii) TOOLS USED	
3.	DESIGN (i) DATA FLOW DIAGRAM (ii) ER DIAGRAM (iii) TABLE DESIGN (iv) ANY UML DIAGRAM	
4.	MODULES MODULE DESCRIPTION (i) Home Page (ii) Creating Profile (iii) Posting Skills (iv) Portal for Officials	
5.	TESTING	
	CONCLUSION	
	APPENDIX (i) SAMPLE CODE (ii) SCREENSHOTS	

CHAPTER 1

PROBLEM STATEMENT

In most academic institutions, information about students' skills, interests, and technical competencies exists only informally — scattered across resumes, personal conversations, or department records that are not searchable or centrally accessible. When faculty, placement coordinators, or event organizers need to identify students with a specific skill (for example, a workshop needing students who know Python, or a hackathon team needing a UI designer), there is no quick or reliable way to search for them. The process typically depends on word-of-mouth referrals or manually going through individual student files, which is slow, inconsistent, and easy to overlook qualified students.

The consequences of this gap extend beyond simple inconvenience. Talented students may never be considered for internships, hackathons, or research assistantships simply because no one in a position to offer the opportunity is aware that the skill exists within the student body. Conversely, faculty and placement coordinators spend disproportionate time manually cross-checking resumes or emailing multiple departments to locate a single qualified candidate, time that could otherwise be spent on higher-value coordination and mentoring activities.

At the same time, students themselves have no single platform to formally register their profile and skill sets in a way that is visible to the institution. Their skills remain siloed in personal portfolios or social profiles that departments do not track, so a student's abilities are effectively invisible to the people who could connect them with opportunities.

This problem tends to worsen as the institution grows. A department with a few dozen students can, at least in principle, be tracked informally by a single coordinator who remembers most names and abilities. However, as student strength increases across multiple departments and academic years, this informal memory-based approach breaks down entirely, and the absence of a structured record becomes a genuine operational bottleneck rather than a minor inconvenience.

This creates two connected problems: students lack a simple way to showcase and update their skills, and administrators/officials lack a search able, centralized system to discover students by skill when opportunities arise. There is no lightweight, purpose-built tool that lets a student register once, add or update skills as they learn them, and lets an official instantly filter and locate the right student — without relying on spreadsheets, manual queries, or informal networks.

Addressing this gap is particularly relevant now, as academic institutions increasingly rely on digital systems for admissions, attendance, and examinations, yet skill and competency data remains one of the few areas still managed manually.

A lightweight, dedicated platform closes this gap without requiring the overhead of a large enterprise system, making it realistic to adopt even within a single department or as a pilot before wider **institutional rollout**.

SkillShare Hub is proposed to address this gap. It is a full-stack web application that lets students register their academic and personal details along with their skill sets, and provides officials with a searchable portal to view, filter, and manage student and skill records — all through a simple, centralized interface backed by a structured database, removing the inefficiency and unreliability of the current informal approach.

The scope of this project is deliberately bounded: it does not attempt to model complex organizational hierarchies, multiple administrator roles, or fine-grained permissions. These boundaries were set so that a complete, working system could be delivered within the capstone timeframe, rather than a partially implemented system attempting to cover a much broader set of requirements.

CHAPTER - 2

SYSTEM STUDY

(i) REVIEW ON THE EXISTING WORK

Introduction

In higher education institutions, student information is generally maintained through different academic and administrative systems. These systems commonly store details such as student names, roll numbers, departments, academic years, attendance, examination records, and other institutional information. However, information related to students' **technical skills, interests, competencies, and additional abilities** is often not maintained in a structured and easily searchable manner.

The increasing importance of employability and skill-based education has created a need for institutions to understand not only what students study, but also what they are capable of doing. Recent literature on digital transformation in education identifies **skills identification** and the use of digital technologies for education and workforce preparation as important areas of study.

Traditional student information systems are useful for maintaining academic and administrative records, but they are not necessarily designed to provide a dedicated view of a student's skills. A student may possess skills such as Python programming, web development, graphic design, communication, database management, or UI/UX design without these abilities being represented in the institution's primary student records.

As a result, institutions have explored different approaches for recording and presenting student competencies.

These approaches include student information systems, learning management systems, electronic portfolios, skill-based profiles, competency-based assessment systems, and placement-oriented platforms. Each approach provides useful functionality.

Traditional Student Record Management

The most common approach used by educational institutions is the **Student Information System (SIS)**. Such systems are primarily designed to maintain structured academic and administrative information. They may contain student identification details, department information, course details, examination records, and other institutional data.

Student information systems provide an important centralized source of student data. Research on higher-education data systems shows that student information systems can provide demographic and academic information that can later be combined with information from other educational systems.

However, traditional student record systems generally focus on **academic administration rather than skill discovery**. For example, a student record may contain:

- Roll number
- Student name
- Department
- Academic year
- Email address
- Course or programme information
- Academic performance

These details are useful for administration, but they do not necessarily answer questions such as:

- Which students know Python?
- Which students have web-development skills?
- Who has experience in UI design?
- Which students can participate in a technical workshop?
- Which students are suitable for a particular hackathon team?

When such information is not maintained in the system, officials may have to depend on resumes, spreadsheets, departmental records, or communication with students and faculty members.

Learning Management Systems

Learning Management Systems (LMS) are another important category of educational software. LMS platforms are primarily used for delivering learning materials, conducting assessments, managing assignments, communicating with students, and tracking learning activities.

Modern educational systems also generate large amounts of student-related data. Research in higher education has explored learning analytics using information obtained from systems such as student information systems and learning platforms.

An LMS can therefore provide valuable information about student participation and learning activities. However, its main purpose is **learning management**, rather than maintaining a simple institutional skill directory.

For example, completing a programming course does not necessarily mean that a student has formally registered Python as a skill. Similarly, participating in a design assignment may indicate experience with design tools, but this information may not be available as a searchable student attribute.

Thus, an LMS is highly useful for learning delivery and academic activity management, but a separate skill-oriented mechanism may still be required when an institution wants to **identify students based on their declared skill**.

Electronic Portfolios

The concept of **electronic portfolios or e-portfolios** provides a more skill-oriented approach. An e-portfolio is a digital collection where students can present their learning achievements, projects, experiences, skills, and evidence of their capabilities.

E-portfolios have been studied as a method for helping students identify and demonstrate employability-related skills. They can connect learning experiences with practical activities and provide evidence of achievements.

Research has also examined the use of e-portfolios for improving the portability of employability skills. One study reported that e-portfolios can be used to encourage students to reflect on and provide evidence of their skill development.

A typical e-portfolio may contain:

- Student introduction
- Academic background
- Technical skills
- Soft skills
- Projects
- Certifications
- Achievements
- Work samples
- Reflections
- Career interests
-

This approach provides considerably more information than a traditional student record.

However, e-portfolios are generally designed around the **individual student's presentation and development**. Their primary purpose is often to allow students to showcase their abilities rather than provide administrators with a lightweight institutional directory for quickly locating students by a particular skill.

For example, if an event coordinator needs five students with Python skills, reviewing individual portfolios can still be time-consuming if the system does not provide an appropriate centralized search mechanism.

Recent work on longitudinal skills portfolios similarly emphasizes maintaining a digital record of skills development over time, including self-assessment, evidence and feedback.

Therefore, e-portfolios demonstrate the value of digital skill records, but they can be more extensive than what is required for a lightweight departmental skill-management application.

Placement and Career-Oriented Platforms

Placement management systems and career platforms also provide mechanisms for recording student skills. These platforms may combine student profiles, academic information, resumes, job opportunities, applications, and placement activities.

Such systems are particularly useful when the primary objective is **recruitment and placement**. Students can create profiles containing their educational qualifications, skills, projects, certifications and other information.

However, the requirements of a general institutional skill directory are somewhat different from those of a placement system.

A placement system primarily focuses on:

- Job opportunities
- Company information
- Applications
- Interviews
- Recruitment
- Placement statistics
- Resumes

A skill-management system focuses more directly on:

- Who possesses a particular skill?
- Which students are available for an activity?
- Which students can participate in a workshop?
- Which students can form a project or hackathon team?
- How can officials quickly identify students based on their skills?

Therefore, a large placement platform may provide more functionality than necessary for the specific problem addressed by SkillShare Hub.

(ii) PROPOSED WORK

To overcome the limitations identified in the existing systems, **SkillShare Hub** is proposed as a lightweight, centralized web-based application for managing and discovering student skills. The main purpose of the system is to make students' skills easily accessible to the people within an institution who need to identify suitable students for academic, technical, extracurricular, or career-related opportunities.

The proposed system combines **student profile management and skill management** into a single application. Instead of depending on separate files, personal resumes, informal communication, or manually maintained records, student information can be stored in a structured database. Students can register their details along with their primary skill, and additional skills can be associated with an existing student.

The system is designed to remain simple and practical. It does not attempt to replace a complete college management system or a large placement platform. Instead, it focuses on one specific requirement: **making student skills visible, searchable, and manageable**.

Overview of SkillShare Hub

SkillShare Hub is developed as a **full-stack web application** consisting of a frontend, backend, and database.

The frontend provides the interface through which users can perform different operations. It contains separate pages for the home page, student registration, student management, adding skills, managing skills, and viewing an individual student's profile. The home page provides navigation to these major functions.

The backend is developed using **FastAPI** and provides REST-style API endpoints for communicating between the frontend and database. The application also uses CORS middleware so that the frontend can communicate with the backend during development.

The database layer is implemented using **MySQL with SQLAlchemy**. SQLAlchemy is used to establish the database connection and create database sessions for handling operations.

The overall working of the proposed system can be represented as:

Student / Official → Frontend → FastAPI Backend → SQLAlchemy → MySQL Database

When a user performs an operation, the frontend sends a request to the backend. The backend processes the request and performs the required database operation. The result is then returned to the frontend and displayed to the user.

Student Registration and Profile Creation

The first major functionality of the proposed system is **student registration**. A student can provide basic information such as roll number, full name, department, academic year, email address, and primary skill. The registration interface contains fields for all these details.

After entering the information, the frontend performs basic validation before sending the data to the backend. It checks whether the required fields have been entered and also performs a simple email validation. The information is then sent to the `/StudentsDetails` API using a POST request.

The backend receives the information through a Pydantic model. The `Details` model defines the expected student attributes: roll number, name, department, year, email, and skill. The email field uses `EmailStr`, providing validation for the email value.

Once the request reaches the backend, the student information is inserted into the `Students_info` database table. The backend commits the transaction so that the new student record is stored in the database.

This approach provides a structured way of creating student profiles rather than maintaining individual records manually.

Skill Registration and Management

A major feature that differentiates SkillShare Hub from a basic student information system is its dedicated **skill management functionality**.

After a student has been registered, an additional skill can be associated with the student's roll number. The Add Skill page asks for the student's roll number and the skill to be added.

The frontend sends this information to the `/Skills` endpoint as a POST request.

The backend stores the information in a separate `Skill_Sets` table. The table contains the student's roll number and skill, with the roll number connected to the `Students_info` table through a foreign key.

This separation makes skill information easier to manage. The system can maintain the student's main profile while also maintaining a dedicated record of skills.

The proposed system also provides a **Manage Skills** page. Officials can view the available skill records, search for a skill, and update a skill when necessary. The page provides a search field and a "Show All" option.

The search operation sends the entered skill name to the backend, which searches the `Skill_Sets` table.

This is one of the most important features of SkillShare Hub because it directly addresses the central problem identified in the problem statement: **finding students according to their skills**.

Centralized Student Management

The proposed system provides a separate **Manage Students** section for viewing and maintaining student records. Instead of checking individual files, officials can view registered students in a structured table.

The table displays information including roll number, name, department, year, email, and skill. It also provides options to update or delete student records.

The backend provides an `/Allstudentinfo` endpoint to retrieve all student records.

The update functionality allows an existing student's information to be modified. The backend receives the updated details and changes the corresponding database record.

The delete functionality removes the selected student record and its corresponding skill record.

Therefore, the proposed system provides the basic **CRUD operations** required for maintaining student information:

- **Create** – Register a student.
- **Read** – View student records.
- **Update** – Modify student details.
- **Delete** – Remove student records.

(iii) TOOLS USED

The development of **SkillShare Hub** requires a combination of frontend, backend, and database technologies. Each technology performs a specific role in the application, from creating the user interface to processing requests and storing student information.

The project follows a **full-stack web application architecture**. The frontend is developed using **HTML, CSS, and JavaScript**, while the backend is developed using **Python and FastAPI**. **SQLAlchemy** is used as the Object Relational Mapping (ORM) layer for communication with the **MySQL** database.

The selected technologies are lightweight and suitable for developing a college-level web application. They also allow the different components of the system to communicate through API requests.

The major tools and technologies used in SkillShare Hub are:

1. HTML
2. CSS
3. JavaScript
4. Python
5. FastAPI
6. SQLAlchemy
7. MySQL

Python

Python is used for developing the backend of SkillShare Hub.

Python is responsible for implementing the application logic that receives requests from the frontend, processes them, communicates with the database, and sends responses back to the frontend.

The backend is divided into separate Python files for different responsibilities:

- `database.py`
- `model.py`
- `table.py`
- `execution.py`
- `end.py`

This separation helps organize the backend into different components.

The project uses Python with FastAPI for creating the web APIs. The backend imports the database, table, and model components before defining its API routes.

Role of Python in SkillShare Hub

Python is used to:

- Implement backend logic.
- Process API requests.
- Validate and handle submitted data.
- Communicate with the database through SQLAlchemy.
- Perform student and skill operations.
- Return responses to the frontend.

Python therefore forms the main **programming language of the backend**.

FastAPI

FastAPI is the web framework used to build the backend API of SkillShare Hub.

The application creates a FastAPI instance and uses API routes to handle operations requested by the frontend.

FastAPI provides endpoints for the major operations of the project.

Some important endpoints include:

HTTP Method	Endpoint	Purpose
POST	/StudentsDetails	Register a student
POST	/Skills	Add a skill
GET	/Allstudentinfo	Retrieve all students
GET	/Skillstudentinfo	Retrieve skill records
GET	/Skillstudentinfo/{ques}	Search skills
GET	/parstudentinfo/{id}	Find a student by roll number
PUT	/Update/{id}	Update student details
PUT	/UpdateSkill/{id}	Update a skill
DELETE	/DeleteStudentsinfo/{id}	Delete student information

The student registration endpoint receives the submitted student details and adds them to the database.

The backend also provides GET endpoints for retrieving all student and skill information.

CORS

FastAPI is also configured with **CORS (Cross-Origin Resource Sharing)** middleware. The project allows the frontend to communicate with the backend during development.

Role of FastAPI

FastAPI acts as the **bridge between the frontend and database**.

The basic flow is:

Frontend → FastAPI API → Database

When the frontend sends a request, FastAPI receives it, processes the request, performs the required operation, and returns the result.

SQLAlchemy

SQLAlchemy is used as the database interaction layer in the project.

Instead of writing every database operation directly as SQL statements, SQLAlchemy allows the Python application to work with database tables through Python classes and objects.

The project creates a SQLAlchemy database engine and session using the configured database connection.

The database tables are represented using Python classes. For example, the `Table` class represents the `Students_info` table, while the `Skill` class represents the `Skill_Sets` table.

The backend uses database sessions to perform operations such as:

- Insert
- Select
- Update
- Delete

For example, the registration API creates a database object and commits it to store a new student.

Role of SQLAlchemy

SQLAlchemy provides:

- Database connectivity.
- Table mapping.
- Database sessions.
- Query operations.
- Insert operations.
- Update operations.
- Delete operations.
- Interaction between Python and MySQL.

Therefore, SQLAlchemy acts as the **ORM/database access layer** of the application.

CHAPTER - 3

(i) DATA FLOW DIAGRAM

Introduction

A **Data Flow Diagram (DFD)** is a graphical representation of how data moves through a system. It shows the interaction between the users, processes, data stores, and the information flowing between them.

For **SkillShare Hub**, the DFD represents how student and skill information moves from the user interface to the backend and database and how the requested information is returned to the user.

The main entities and processes involved are:

- **Student** – Provides personal details and skills.
- **Official** – Views, searches, updates, and manages student information.
- **Frontend** – Provides forms, search pages, tables, and profile pages.
- **FastAPI Backend** – Processes requests and performs the required operations.
- **MySQL Database** – Stores student and skill information.

The frontend communicates with the FastAPI backend through API requests. The backend uses SQLAlchemy to perform database operations. The project contains separate database tables for student information and skill information.

(i) Data Flow Diagram

The DFD illustrates how student and official interactions move through the system: students feed profile and skill data into the registration process, which is stored in the two database tables; officials query those same tables through the search/view/update/delete process.

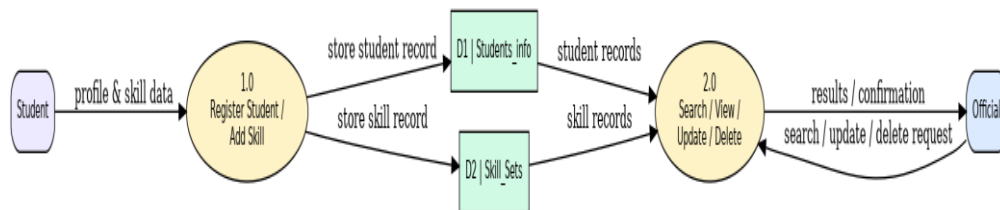


Fig 3.1 — Data Flow Diagram

(ii) ER Diagram

Students_info and Skill_Sets are related through rollno — a student can have zero or one additional skill record in the current implementation (1 : 0..1), with Skill_Sets.rollno acting as both the primary key and foreign key referencing Students_info.rollno.

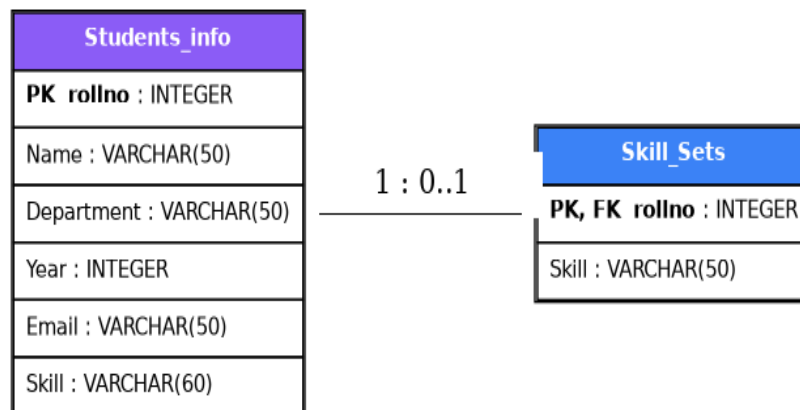


Fig 3.2 — Entity Relationship Diagram

(iii) Table Design

Students_info:

Column	Type	Constraint
rollno	INTEGER	Primary Key
Name	VARCHAR(50)	No explicit NOT NULL constraint
Department	VARCHAR(50)	No explicit NOT NULL constraint
Year	INTEGER	No explicit NOT NULL constraint
Email	VARCHAR(50)	No explicit NOT NULL constraint
Skill	VARCHAR(60)	No explicit NOT NULL constraint

Skill_Sets:

Column	Type	Constraint
rollno	INTEGER	Primary Key, Foreign Key → Students_info.rollno
Skill	VARCHAR(50)	No explicit NOT NULL constraint

(iv) UML Diagram (Use Case)

Two actors interact with the system: the Student, who registers a profile and posts skills, and the Official, who views, searches, updates, and deletes student/skill records.

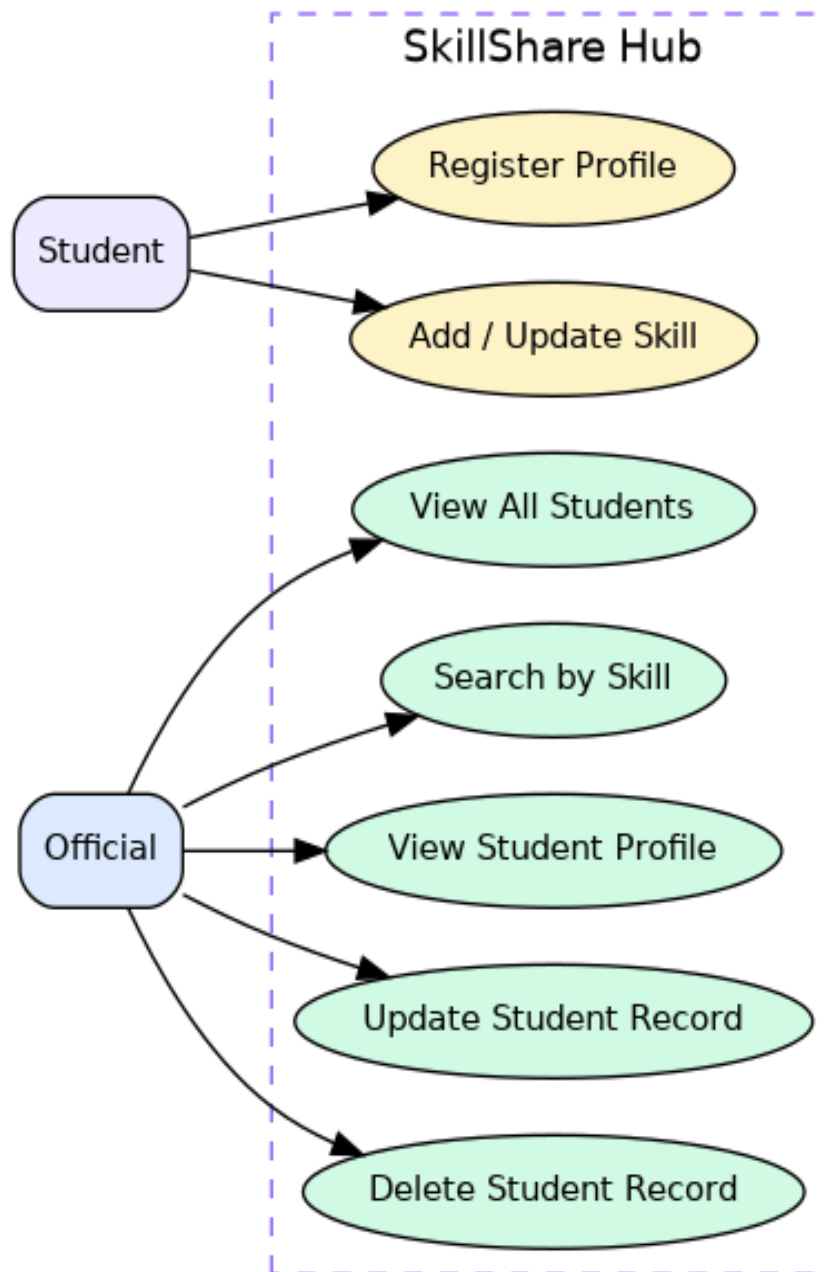


Fig 3.3 — Use Case Diagram

CHAPTER - 4

MODULES

(i) Home Page

HOME PAGE

The **Home Page** is the main interface of the **SkillShare Hub** application. It acts as the starting point for users and provides access to all the major functionalities of the system. The page is designed to give users a clear understanding of the purpose of the application and allow them to quickly navigate to the required module.

The Home Page displays the project name “**SkillShare Hub**” along with the tagline “**Discover • Learn • Connect**”, which represents the main purpose of the application.

The page provides five main navigation options:

- **Register Student**
- **Manage Students**
- **Add Skill**
- **Manage Skills**
- **Student Profile**

These options are presented as separate cards, making it easy for users to identify and access the required functionality.

Register Student

The **Register Student** option takes the user to the student registration page. It is used to create a new student profile by entering details such as roll number, name, department, academic year, email, and primary skill.

Manage Students

The **Manage Students** option provides access to the student management section. This section allows the available student records to be viewed and provides options for updating and deleting student information.

Add Skill

The **Add Skill** option allows a new skill to be associated with an existing student using the student's roll number. This is useful when a student wants to add another skill after creating their profile.

Manage Skills

The **Manage Skills** option provides access to skill records. Users can search for a particular skill, display all available skills, and update skill information.

Student Profile

The **Student Profile** option allows a student record to be searched using the roll number. The system displays the corresponding student's name, department, year, email, and skill.

Design of the Home Page

The Home Page uses a card-based layout to organize the available functions. The cards use icons, titles, and short descriptions to explain their purpose. The frontend also includes visual effects such as hover animations and gradient styling to make the interface more interactive.

The common stylesheet provides the overall visual design of the application, including the navigation bar, cards, colors, typography, shadows, and layout.

(ii) Creating Profile

The **Creating Profile** module is used to register a student in the SkillShare Hub system. It provides a simple form through which the student's basic academic and contact information can be entered and stored in the database. This module is important because the student information entered here becomes part of the centralized student records that can later be viewed, managed, and searched.

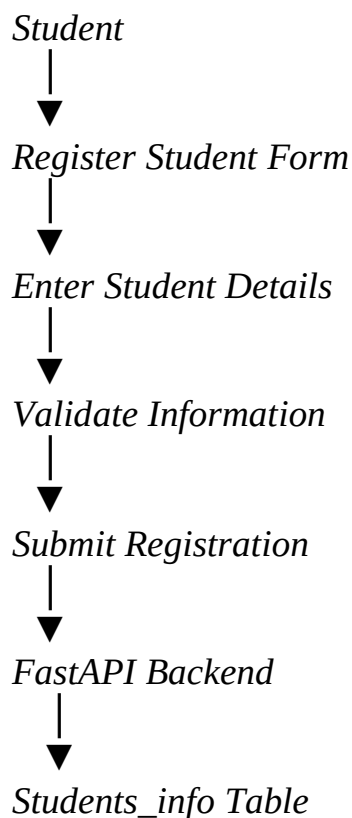
The registration form collects the student's **Roll Number, Name, Department, Year, Email, and Primary Skill**. These fields provide the essential information required to create a student profile.

Profile Creation Process

The student first opens the **Register Student** page from the Home Page. The required details are entered into the corresponding fields. The application performs basic validation to ensure that the required information is provided and that the email address follows the expected format.

After entering the details, the student clicks the **Register Student** button. The frontend sends the entered information to the backend through the `/StudentsDetails` API endpoint. The backend receives the data and creates a new student record in the database.

The student information is stored in the **Students_info** database table. The table contains fields for roll number, name, department, year, email, and skill.





Student Profile Created

Importance of the Module

The Creating Profile module provides a structured way of collecting student information instead of maintaining records manually. Once a profile is created, the stored information can be used by other modules such as **Manage Students** and **Student Profile**. This makes the student information easier to maintain and retrieve within the SkillShare Hub application.

Overall, the module provides a straightforward registration process and forms the **foundation for creating centralized student profiles** in the system.

(iii) Posting Skills

The **Posting Skills** module allows students to add their skills to the SkillShare Hub system. It is useful for maintaining an updated record of the technical or other skills possessed by each student. A skill can be associated with a student using the student's **Roll Number**.

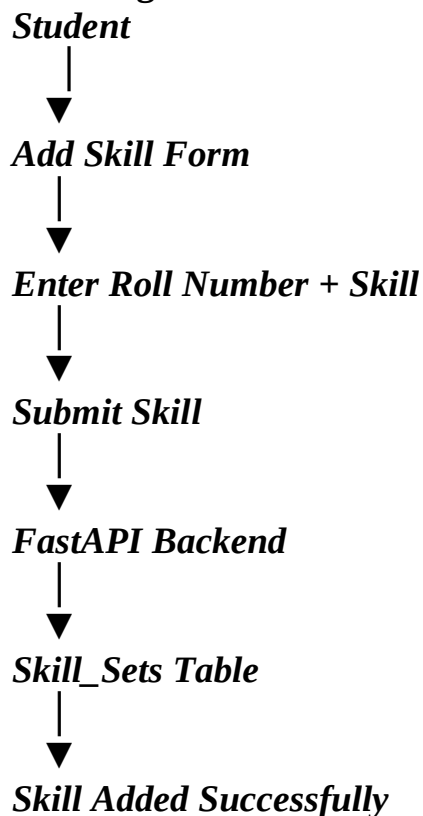
The module provides a simple form containing two main fields: **Roll Number** and **Skill**. The user enters the roll number of the student and specifies the skill that needs to be added.

Skill Posting Process

After entering the required details, the user selects the **Add Skill** option. The frontend sends the entered roll number and skill information to the backend through the `/Skills` API. The backend receives the information and stores it as a new skill record.

The skill information is stored in the **Skill_Sets** table. The table contains the student's roll number and the corresponding skill, with the roll number connected to the student record through a foreign key.

Working Flow



Skill Management

The posted skills can later be viewed and managed through the **Manage Skills** module. The system provides functionality to search for skills, display available skill records, and update an existing skill.

This module therefore helps maintain a centralized collection of student skills and makes the information available for searching and management within the SkillShare Hub system. It provides a simple method for keeping student skill information updated and organized.

(iv) Portal for Officials

The Portal for Officials is an important part of the SkillShare Hub application that helps official users manage student and skill information from a centralized interface. It provides access to the Manage Students and Manage Skills functionalities, allowing student records and skill records to be viewed, updated, and managed efficiently.

Student Management

Through the **Manage Students** section, officials can view the available student records in a table format. The information includes the student's **Roll Number, Name, Department, Year, Email, and Skill**. Officials can also edit existing student information or delete a student record when required.

The backend supports these operations through APIs for retrieving all student records, updating student information, and deleting student information.

Skill Management

The portal also provides a **Manage Skills** section. Officials can view the available skill records and search for a particular skill. The module also provides an option to update an existing skill when changes are required.

The backend provides separate API operations for retrieving skill information, searching skills, and updating skill records.

The Portal for Officials therefore provides a **centralized management interface** for maintaining student and skill information. Instead of handling records separately, officials can access the required information through the application and perform the necessary management operations. This improves the organization of records and makes student skill information easier to maintain within the SkillShare Hub system.

CHAPTER – 5

TESTING

TESTING, CONCLUSION, APPENDIX

- i) SAMPLE CODE
- ii) SCREENSHOTS

Testing was carried out at multiple levels to ensure that **SkillShare Hub functions correctly from end to end**. The testing process covered the FastAPI backend, database operations, API endpoints, and browser-based frontend. The main objective was to verify that data entered through the frontend is correctly processed by the backend and stored, retrieved, updated, or deleted from the database as expected.

Testing also helped identify incorrect inputs and verify that the application provides appropriate responses when invalid data or non-existent records are provided.

Types of Testing Performed

1. Unit Testing

Each API endpoint was tested independently using **FastAPI's built-in Swagger UI (/docs)**. This helped verify individual backend operations such as registering students, adding skills, retrieving records, updating information, and deleting records. Both valid and invalid inputs were considered wherever applicable.

2. Integration Testing

Integration testing was performed to verify communication between the **frontend and backend**. The browser pages use the **Fetch API** to send requests to the FastAPI endpoints. The responses received from the backend were checked to ensure that the frontend displayed the appropriate results, success messages, or error messages.

3. System Testing

The complete application was tested as an integrated system. The major workflow was tested in sequence:

Register a student → Add a skill → Search/View information → Update information → Delete student

This confirmed that the different modules work together properly and that changes made through one operation are reflected in subsequent operations.

4. UI Testing

Manual UI testing was performed across the six frontend pages. The testing included form validation, toast notifications, modal behaviour, search functionality, and empty-state displays. The objective was to ensure that the interface responds properly to user actions and provides understandable feedback.

Test Cases

Test Case	Endpoint	Input	Expected Result	Actual Result	Status
Register a new student	POST /StudentsDetails	Valid rollno, name, dept, year, email, skill	Record inserted, success message	"Added success" returned	Pass
Register with invalid email	POST /StudentsDetails	Malformed email (no @)	Validation error from Pydantic EmailStr	422 error returned	Pass
Add a skill to existing student	POST /Skills	Valid rollno + skill	Row inserted into Skill_Sets	"Added Successfully" returned	Pass
Add skill for non-existent student	POST /Skills	rollno not in Students_info	Rejected due to FK constraint	Insert fails (FK violation)	Pass
View all students	GET /Allstudentinfo	—	Full list of student records	List returned correctly	Pass
View all skills	GET /Skillstudentinfo	—	Full list of skill records	List returned correctly	Pass
Search by skill	GET /Skillstudentinfo/{ques}	e.g. "python"	Matching records (case-insensitive)	Correct filtered list returned	Pass
Fetch student by rollno	GET /parstudentinfo/{id}	Valid rollno	Student's full profile	Correct record returned	Pass
Fetch with invalid rollno	GET /parstudentinfo/{id}	Non-existent rollno	Handled gracefully in UI	"Student not found" shown	Pass
Update student record	PUT /Update/{id}	Valid rollno + updated fields	Record updated	"Status Updated" returned	Pass
Update with invalid id	PUT /Update/{id}	Non-existent rollno	Error, no update performed	"Problem in entered Id" returned	Pass
Update skill record	PUT /UpdateSkill/{id}	Valid rollno + new skill	Skill updated	"Updated" returned	Pass
Delete student	DELETE /DeleteStudentsinfo/{id}	Valid rollno	Student & linked skill removed	"Deleted", both rows removed	Pass
Delete with invalid id	DELETE /DeleteStudentsinfo/{id}	Non-existent rollno	Error, nothing deleted	"Problem in entered Id" returned	Pass

Testing Outcome

The testing process confirmed that the major operations of SkillShare Hub work as intended. **Student registration** successfully creates records, while invalid email input is rejected through validation. **Skill posting** correctly stores skill information for existing students, and the foreign-key relationship prevents skills from being associated with non-existent student records.

The retrieval and search operations were also verified successfully. Student and skill records could be displayed correctly, and skill searches returned matching records using case-insensitive filtering. Update and delete operations were tested with both valid and invalid identifiers to ensure that valid records are modified or removed while invalid requests do not unintentionally affect existing data.

The testing also verified the interaction between the **frontend, FastAPI backend, and database layer**. Appropriate success and error messages were displayed in the user interface, making the system easier to operate and understand.

Overall Result

All core functionalities of **SkillShare Hub** — **registration, skill posting, viewing, searching, updating, and deletion** — **passed their respective test cases**. The results indicate that the application components interact reliably and that the implemented functionality meets the intended requirements of the project.

CONCLUSION

The **SkillShare Hub** project successfully addresses the problem of fragmented and difficult-to-search student skill information within an academic institution. By replacing informal and manual methods of maintaining student skill records with a centralized full-stack web application, the system provides students with a simple platform to register their profiles and add their skills. At the same time, officials can access a centralized portal to view, search, update, and manage student and skill information efficiently.

The application was developed using **FastAPI** for the backend, **SQLAlchemy** for database interaction, **Pydantic** for request validation, and **MySQL** for storing the application data. The frontend was developed using **HTML, CSS, and JavaScript** and communicates with the backend through REST API endpoints. The database consists of the `Students_info` and `Skill_Sets` tables, with the skill records linked to student records through a foreign-key relationship. This structure helps maintain consistency between student and skill information.

The project successfully implements the four major modules: **Home Page, Creating Profile, Posting Skills, and Portal for Officials**. The system supports the major operations required for the project, including student registration, skill posting, viewing records, searching skills, updating information, and deleting student records. These functionalities were tested at the API, integration, system, and user-interface levels, and the core test cases produced successful results.

Apart from developing the application, the project provided practical experience in **full-stack web development**. It involved designing a relational database, creating and testing REST APIs, connecting the frontend with backend services, handling user input and validation, and implementing database operations through an ORM. The project therefore demonstrates how a simple web-based system can be used to organize student skill information and make it more accessible for academic purposes.

Overall, SkillShare Hub provides a functional prototype that meets its intended objectives and establishes a foundation for further development. With additional security, scalability, and skill-management features, the application can be extended into a more comprehensive institutional platform.

Future Enhancements

The following improvements can be considered in future versions of SkillShare Hub:

- **Authentication and Role-Based Access:** Add secure login functionality so that students and officials can access appropriate role-specific features.

- **Multiple Skills and Proficiency Levels:** Allow students to maintain multiple skills and specify their proficiency level for each skill instead of relying on a single free-text skill entry.
- **Advanced Search and Filtering:** Introduce pagination and filters based on department, academic year, and other relevant criteria to make searching more efficient as the number of student records increases.
- **Cloud Deployment:** Deploy the application to a cloud platform with a production-grade database instead of relying on a locally hosted MySQL database, making the system accessible and scalable for institutional use.

Final Statement

Thus, **SkillShare Hub successfully demonstrates a centralized and practical approach to student skill management.** The completed prototype provides the essential functionality required to register, store, search, update, and manage student skill information while also providing a strong foundation for future enhancements.

APPENDIX

APPENDIX

i) Sample Code

Database Configuration — database.py

```
from sqlalchemy.orm import sessionmaker
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import
declarative_base

url =
"mysql+pymysql://root:****@localhost:3306/project"
engine = create_engine(url)
Session = sessionmaker(autocommit=False,
autoflush=False, bind=engine)

def injection():
    db = Session()
    try:
        yield db
    finally:
        db.close()
```

```
Base = declarative_base()
```

Table Models — table.py

```
from database import Base
from sqlalchemy import Column, INTEGER, VARCHAR,
ForeignKey

class Table(Base):
    __tablename__ = "Students_info"
    rollno = Column(INTEGER, primary_key=True)
    Name = Column(VARCHAR(50))
    Department = Column(VARCHAR(50))
    Year = Column(INTEGER)
    Email = Column(VARCHAR(50))
    Skill = Column(VARCHAR(60))

class Skill(Base):
    __tablename__ = "Skill_Sets"
    rollno = Column(INTEGER,
ForeignKey("Students_info.rollno"),
primary_key=True)
    Skill = Column(VARCHAR(50))
```

Pydantic Schemas — model.py

```
from pydantic import BaseModel, EmailStr

class Details(BaseModel):
    rollno: int
    Name: str
    Department: str
    Year: str
    Email: EmailStr
    Skill: str
```

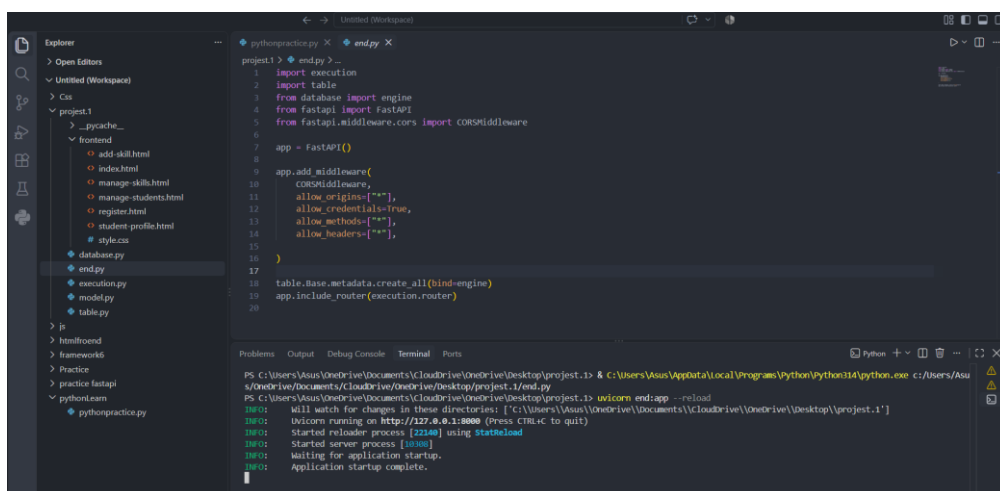
```
class Gskills(BaseModel):
    rollno: int
    Skill: str
```

Sample Endpoint — execution.py

```
@router.post("/StudentsDetails")
def details(content: model.Details, db: Session = Depends(injection)):
    db.add(table.Table(**content.model_dump()))
    db.commit()
    return "Added success"

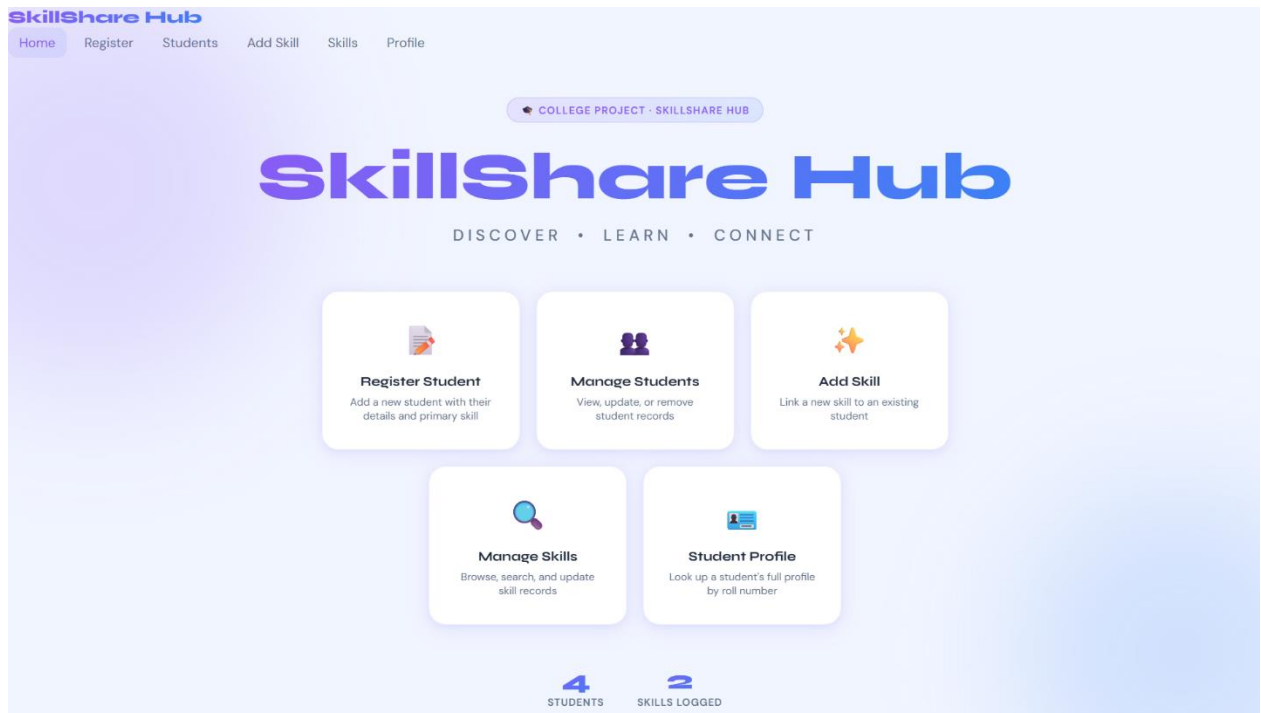
@router.get("/Skillstudentinfo/{ques}")
def ssinfo(ques: str, db: Session = Depends(injection)):
    emp =
db.query(table.Skill).filter(table.Skill.Skill.ilike(ques)).all()
```

ii) Screenshots



Screenshot 1 — Backend server running via Uvicorn in VS Code (end.py), served at <http://127.0.0.1:8000>

1.HOME PAGE



2.REGISTRATION

The screenshot shows the SkillShare Hub registration page. At the top, there is a navigation bar with links: HOME, REGISTER, STUDENTS, ADD SKILL, SKILLS, and PROFILE. The main heading is "Register Student" in large blue letters, followed by the instruction "Fill in the form below to add a new student to SkillShare Hub". Below this, there is a registration form with the following fields:

- ROLL NUMBER: 106
- FULL NAME: Hariniya Raj
- DEPARTMENT: Bsc CS
- YEAR: 2nd Year
- EMAIL: herini125@gmail.com
- PRIMARY SKILL: AI/ML, Python

At the bottom of the form, there is a blue button labeled "Register Student".

3.STUDENTS SKILL POSTS

SkillShare Hub

HOME

REGISTER

STUDENTS

ADD SKILL

SKILLS

PROFILE

Manage Students

View, edit, or remove student records from the hub

All Students

Refresh

ROLL NO	NAME	DEPARTMENT	YEAR	EMAIL	SKILL	UPDATE	DELETE
101	Jagan.B	BSC-IT	Year 2	jagand131@gmail.com	Python,Java,Sql	Edit	Delete
103	Renuga.B	Bsc CS	Year 2	renu131@gmail.com	Python with restapi, Java, Video Editing.	Edit	Delete
106	Hariniya Raj	Bsc CS	Year 2	herini125@gmail.com	AI/ML, Python	Edit	Delete
166	Gawri	ECE	Year 2	gawrijagal23@gmail.com	C++,JAVA,PYTHON	Edit	Delete

Home

Register

Students

Add Skill

Skills

Profile

4.STUDENTS PROFILES

SkillShare Hub

HOME

REGISTER

STUDENTS

ADD SKILL

SKILLS

PROFILE

ROLL NUMBER103

Search Student

R

Renuga.B

Bsc CS

ROLL NO103

YEARYear 2

EMAILrenu131@gmail.com

SKILLPython with restapi, Java, Video Editing.